

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Name: Humera Nuzhat

H.T No:2303A52083

Batch No: 39

Task 1 – Student Information System Schema

Task:

Design a database schema for a **Student Information System** and generate queries using AI.

```
lab1.sql
Connect to MSSQL | Run on active connection | Select block
1  -- Create tables for Student Information System
2  -- Tables: Students, Courses, Enrollments
3  -- Include primary key and foreign key
4
5  -- Create Students table
6  CREATE TABLE Students (
7      student_id INT PRIMARY KEY,
8      first_name VARCHAR(50) NOT NULL,
9      last_name VARCHAR(50) NOT NULL,
10     email VARCHAR(100) UNIQUE NOT NULL
11 );
12 -- Create Courses table
13 CREATE TABLE Courses (
14     course_id INT PRIMARY KEY,
15     course_name VARCHAR(100) NOT NULL,
16     credits INT NOT NULL
17 );
18 -- Create Enrollments table
19 CREATE TABLE Enrollments (
20     enrollment_id INT PRIMARY KEY,
21     student_id INT,
22     course_id INT,
23     enrollment_date DATE,
24     FOREIGN KEY (student_id) REFERENCES Students(student_id),
25     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
26 );
```

```

27 -- Insert sample data into Students table
28 INSERT INTO Students (student_id, first_name, last_name, email) VALUES
29 (1, 'John', 'Doe', 'john.doe@example.com'),
30 (2, 'Jane', 'Smith', 'jane.smith@example.com');
31 -- Insert sample data into Courses table
32 INSERT INTO Courses (course_id, course_name, credits) VALUES
33 (101, 'Introduction to Computer Science', 3),
34 (102, 'Database Systems', 4);
35 -- Insert sample data into Enrollments table
36 INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date)
VALUES
37 (1, 1, 101, '2023-09-01'),
38 (2, 2, 102, '2023-09-01');
39 -- Query to retrieve all students and their enrolled courses
40 SELECT s.first_name, s.last_name, c.course_name
41 FROM Students s
42 JOIN Enrollments e ON s.student_id = e.student_id
43 JOIN Courses c ON e.course_id = c.course_id;

```

Output:

```

31 -- Insert sample data into Courses table
32 INSERT INTO Courses (course_id, course_name, credits) VALUES
33 (101, 'Introduction to Computer Science', 3),
34 (102, 'Database Systems', 4);
35 -- Insert sample data into Enrollments table
36 INSERT INTO Enrollments (enrollment_id, student_id, course_id, enrollment_date) VALUES
37 (1, 1, 101, '2023-09-01'),
38 (2, 2, 102, '2023-09-01');
39 -- Query to retrieve all students and their enrolled courses
40 SELECT s.first_name, s.last_name, c.course_name
41 FROM Students s
42 JOIN Enrollments e ON s.student_id = e.student_id
43 JOIN Courses c ON e.course_id = c.course_id;

```

first_name	last_name	course_name
John	Doe	Introduction to Computer Science
Jane	Smith	Database Systems

Explanation:

This database uses three tables: Students, Courses, and Enrollments to manage relationships between students and courses. The Enrollments table acts as a bridge using foreign keys, ensuring referential integrity. Insert operations add new student records, while JOIN queries retrieve courses for a student. Aggregation using COUNT helps determine the number of students per course. This design follows normalization principles and avoids data redundancy.

Task 2 – Hospital Management Database

Task:

Create schema and queries for a **Hospital Management System**.

```
1  --generate a Hospital database schema with doctors, patients,  
   appointments and SQL queries with joins  
2  -- Create the Hospital database  
3  CREATE DATABASE HospitalDB;  
4  USE HospitalDB;  
5  -- Create Doctors table  
6  CREATE TABLE Doctors (  
7      doctor_id INT PRIMARY KEY,  
8      first_name VARCHAR(50) NOT NULL,  
9      last_name VARCHAR(50) NOT NULL,  
10     specialty VARCHAR(100) NOT NULL  
11 );  
12 -- Create Patients table  
13 CREATE TABLE Patients (  
14     patient_id INT PRIMARY KEY,  
15     first_name VARCHAR(50) NOT NULL,  
16     last_name VARCHAR(50) NOT NULL,  
17     date_of_birth DATE NOT NULL  
18 );
```

```

19  -- Create Appointments table
20  CREATE TABLE Appointments (
21      appointment_id INT PRIMARY KEY,
22      doctor_id INT,
23      patient_id INT,
24      appointment_date DATE,
25      FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
26      FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
27  );
28  -- Insert sample data into Doctors table
29  INSERT INTO Doctors (doctor_id, first_name, last_name,
30      specialty) VALUES
31  (1, 'Alice', 'Johnson', 'Cardiology'),
32  (2, 'Bob', 'Smith', 'Neurology');
33  -- Insert sample data into Patients table
34  INSERT INTO Patients (patient_id, first_name, last_name,
35      date_of_birth) VALUES
36  (1, 'Charlie', 'Brown', '1980-05-15'),
37  (2, 'Diana', 'Prince', '1990-08-25');
38
39  INSERT INTO Appointments (appointment_id, doctor_id,
40      patient_id, appointment_date) VALUES
41  (1, 1, 1, '2023-10-01'),
42  (2, 2, 2, '2023-10-02');
43
44  -- Query to retrieve all doctors and their patients
45  SELECT d.first_name AS doctor_first_name, d.last_name AS
46      doctor_last_name, p.first_name AS patient_first_name, p.
47      last_name AS patient_last_name
48  FROM Doctors d
49  JOIN Appointments a ON d.doctor_id = a.doctor_id
50  JOIN Patients p ON a.patient_id = p.patient_id;
51
52  -- Query to retrieve all patients and their doctors
53  SELECT p.first_name AS patient_first_name, p.last_name AS
54      patient_last_name, d.first_name AS doctor_first_name, d.
55      last_name AS doctor_last_name
56  FROM Patients p
57  JOIN Appointments a ON p.patient_id = a.patient_id
58  JOIN Doctors d ON a.doctor_id = d.doctor_id;

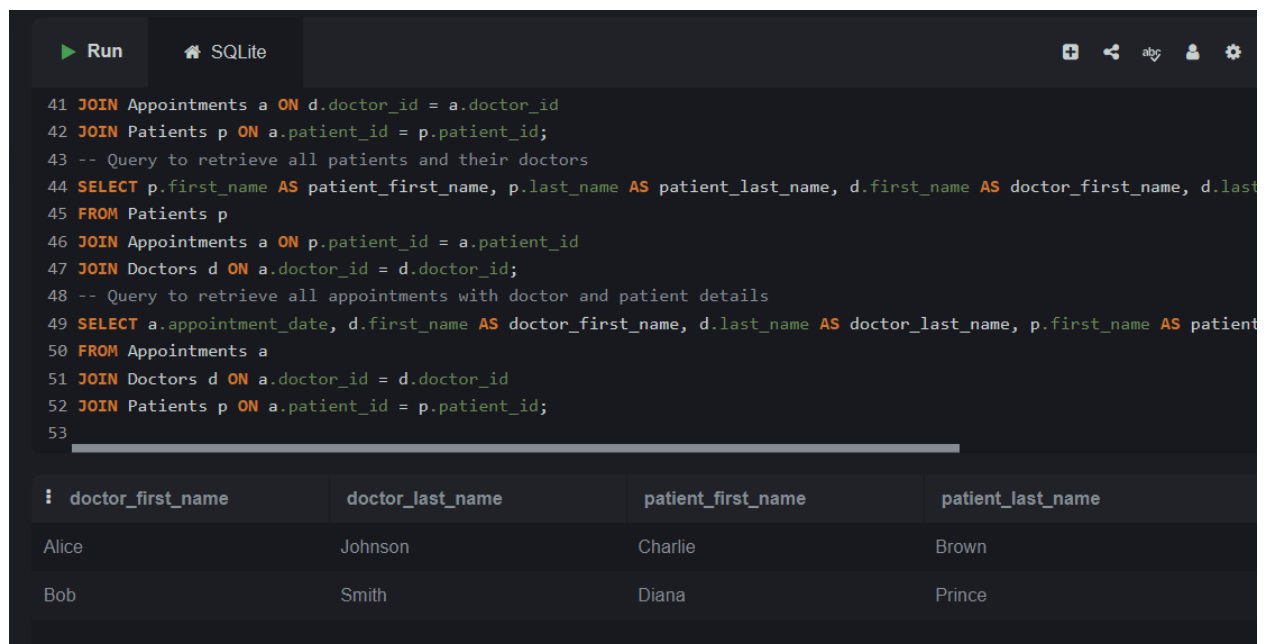
```

```

50 -- Query to retrieve all appointments with doctor and patient
    details
51 SELECT a.appointment_date, d.first_name AS doctor_first_name, d.
    last_name AS doctor_last_name, p.first_name AS
    patient_first_name, p.last_name AS patient_last_name
52 FROM Appointments a
53 JOIN Doctors d ON a.doctor_id = d.doctor_id
54 JOIN Patients p ON a.patient_id = p.patient_id;
55

```

Output:



The screenshot shows a SQLite IDE interface. The top bar has a 'Run' button and a 'SQLite' label. The main area contains several SQL queries. The bottom section displays the results of the last query in a table format.

```

41 JOIN Appointments a ON d.doctor_id = a.doctor_id
42 JOIN Patients p ON a.patient_id = p.patient_id;
43 -- Query to retrieve all patients and their doctors
44 SELECT p.first_name AS patient_first_name, p.last_name AS patient_last_name, d.first_name AS doctor_first_name, d.last
45 FROM Patients p
46 JOIN Appointments a ON p.patient_id = a.patient_id
47 JOIN Doctors d ON a.doctor_id = d.doctor_id;
48 -- Query to retrieve all appointments with doctor and patient details
49 SELECT a.appointment_date, d.first_name AS doctor_first_name, d.last_name AS doctor_last_name, p.first_name AS patient
50 FROM Appointments a
51 JOIN Doctors d ON a.doctor_id = d.doctor_id
52 JOIN Patients p ON a.patient_id = p.patient_id;
53

```

doctor_first_name	doctor_last_name	patient_first_name	patient_last_name
Alice	Johnson	Charlie	Brown
Bob	Smith	Diana	Prince

Explanation:

The hospital database consists of Doctors, Patients, and Appointments tables. Relationships are maintained using foreign keys to link doctors and patients through appointments. Queries allow retrieval of doctor schedules and patient history efficiently. Aggregation functions count the number of patients handled by each doctor. This structured schema ensures data consistency, reduces duplication, and supports efficient query execution using joins.

Task 3 – Library Database

Task:

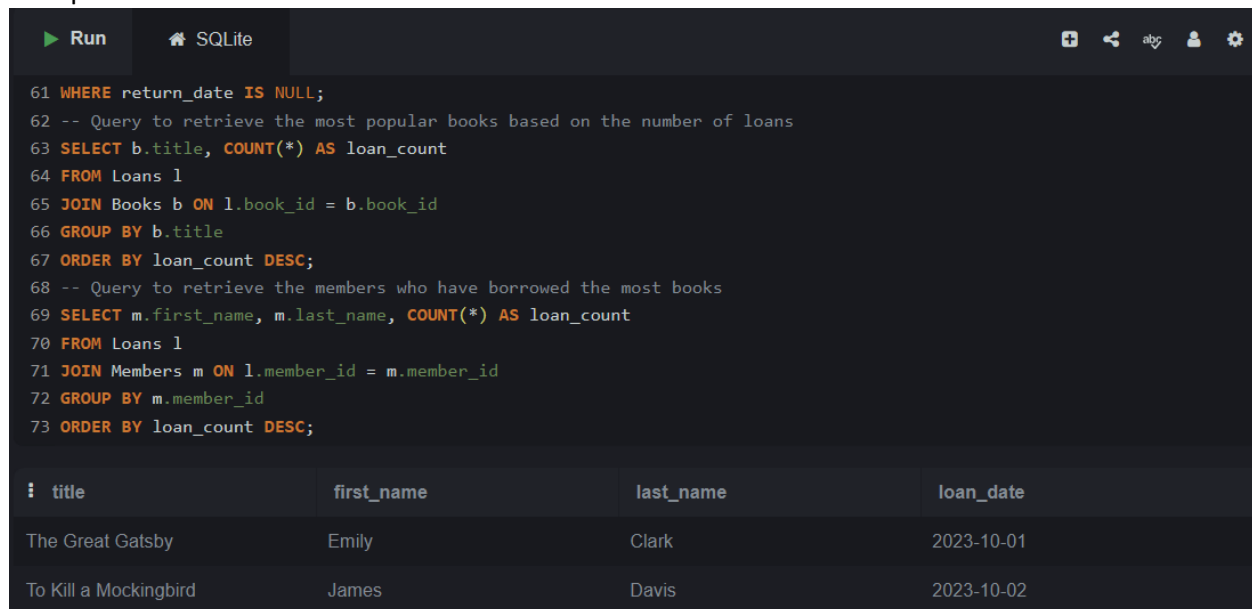
Design schema for a **Library Management System**.

```
1  --generate Library management SQL schema and queries including
   overdue books and indexing
2  -- Create the Library database
3  CREATE DATABASE LibraryDB;
4  USE LibraryDB;
5  -- Create Books table
6  CREATE TABLE Books (
7      book_id INT PRIMARY KEY,
8      title VARCHAR(255) NOT NULL,
9      author VARCHAR(255) NOT NULL,
10     published_date DATE NOT NULL,
11     available_copies INT NOT NULL
12 );
13 -- Create Members table
14 CREATE TABLE Members (
15     member_id INT PRIMARY KEY,
16     first_name VARCHAR(50) NOT NULL,
17     last_name VARCHAR(50) NOT NULL,
18     email VARCHAR(100) UNIQUE NOT NULL
19 );
20 -- Create Loans table
21 CREATE TABLE Loans (
22     loan_id INT PRIMARY KEY,
23     book_id INT,
24     member_id INT,
25     loan_date DATE,
26     return_date DATE,
27     FOREIGN KEY (book_id) REFERENCES Books(book_id),
28     FOREIGN KEY (member_id) REFERENCES Members(member_id)
29 );
30 -- Insert sample data into Books table
31 INSERT INTO Books (book_id, title, author, published_date,
   available_copies) VALUES
32 (1, 'The Great Gatsby', 'F. Scott Fitzgerald', '1925-04-10', 3),
33 (2, 'To Kill a Mockingbird', 'Harper Lee', '1960-07-11', 2);
```

```
35 INSERT INTO Members (member_id, first_name, last_name, email)
VALUES
36 (1, 'Emily', 'Clark', 'emily.clark@example.com'),
37 (2, 'James', 'Davis', 'james.davis@example.com');
38 -- Insert sample data into Loans table
39 INSERT INTO Loans (loan_id, book_id, member_id, loan_date,
return_date) VALUES
40 (1, 1, 1, '2023-10-01', NULL),
41 (2, 2, 2, '2023-10-02', NULL);
42 -- Query to retrieve all books currently on loan
43 SELECT b.title, m.first_name, m.last_name, l.loan_date
44 FROM Loans l
45 JOIN Books b ON l.book_id = b.book_id
46 JOIN Members m ON l.member_id = m.member_id
47 WHERE l.return_date IS NULL;
48 -- Create an index on the loan_date column for faster retrieval
of overdue books
49 CREATE INDEX idx_loan_date ON Loans(loan_date);
50 -- Query to retrieve all members and the books they have
currently on loan
```

```
51 SELECT m.first_name, m.last_name, b.title
52 FROM Members m
53 JOIN Loans l ON m.member_id = l.member_id
54 JOIN Books b ON l.book_id = b.book_id
55 WHERE l.return_date IS NULL;
56 -- Query to retrieve the total number of available copies for
   each book
57 SELECT title, available_copies
58 FROM Books;
59 -- Query to retrieve the total number of books currently on loan
60 SELECT COUNT(*) AS total_loans
61 FROM Loans
62 WHERE return_date IS NULL;
63 -- Query to retrieve the most popular books based on the number
   of loans
64 SELECT b.title, COUNT(*) AS loan_count
65 FROM Loans l
66 JOIN Books b ON l.book_id = b.book_id
67 GROUP BY b.title
68 ORDER BY loan_count DESC;
69 -- Query to retrieve the members who have borrowed the most
   books
70 SELECT m.first_name, m.last_name, COUNT(*) AS loan_count
71 FROM Loans l
72 JOIN Members m ON l.member_id = m.member_id
73 GROUP BY m.member_id
74 ORDER BY loan_count DESC;
```


Output:



The screenshot shows a SQLite IDE interface. At the top, there is a 'Run' button and a 'SQLite' tab. Below the toolbar, there is a text area containing two SQL queries. The first query is a SELECT statement that retrieves the title and loan count from the Loans table, joined with the Books table. The second query is a SELECT statement that retrieves the first and last names of members who have borrowed the most books, joined with the Loans table. Below the queries, there is a table with four columns: title, first_name, last_name, and loan_date. The table contains two rows of data: 'The Great Gatsby' by Emily Clark with a loan date of 2023-10-01, and 'To Kill a Mockingbird' by James Davis with a loan date of 2023-10-02.

```
61 WHERE return_date IS NULL;
62 -- Query to retrieve the most popular books based on the number of loans
63 SELECT b.title, COUNT(*) AS loan_count
64 FROM Loans l
65 JOIN Books b ON l.book_id = b.book_id
66 GROUP BY b.title
67 ORDER BY loan_count DESC;
68 -- Query to retrieve the members who have borrowed the most books
69 SELECT m.first_name, m.last_name, COUNT(*) AS loan_count
70 FROM Loans l
71 JOIN Members m ON l.member_id = m.member_id
72 GROUP BY m.member_id
73 ORDER BY loan_count DESC;
```

title	first_name	last_name	loan_date
The Great Gatsby	Emily	Clark	2023-10-01
To Kill a Mockingbird	James	Davis	2023-10-02

Explanation:

The library system uses Books, Members, and Loans tables to manage book borrowing activities. The Loans table connects books and members, maintaining issue and return details. Queries identify issued books, overdue records, and borrowing counts. Indexing improves query performance for frequent searches. This design supports efficient tracking of library operations while maintaining normalized structure and minimizing redundancy.

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an **E-commerce platform**.

```
1  --generate Ecommerce database SQL schema with users, products,  
   orders and analytics queries  
2  -- Create the Ecommerce database  
3  CREATE DATABASE EcommerceDB;  
4  USE EcommerceDB;  
5  -- Create Users table  
6  CREATE TABLE Users (  
7      user_id INT PRIMARY KEY,  
8      first_name VARCHAR(50) NOT NULL,  
9      last_name VARCHAR(50) NOT NULL,  
10     email VARCHAR(100) UNIQUE NOT NULL  
11 );  
12 -- Create Products table  
13 CREATE TABLE Products (  
14     product_id INT PRIMARY KEY,  
15     product_name VARCHAR(255) NOT NULL,  
16     price DECIMAL(10, 2) NOT NULL,  
17     stock INT NOT NULL  
18 );  
19 -- Create Orders table  
20 CREATE TABLE Orders (  
21     order_id INT PRIMARY KEY,  
22     user_id INT,  
23     order_date DATE,  
24     total_amount DECIMAL(10, 2) NOT NULL,  
25     FOREIGN KEY (user_id) REFERENCES Users(user_id)  
26 );  
27 -- Create OrderItems table  
28 CREATE TABLE OrderItems (  
29     order_item_id INT PRIMARY KEY,  
30     order_id INT,  
31     product_id INT,  
32     quantity INT NOT NULL,  
33     price DECIMAL(10, 2) NOT NULL,  
34     FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
35     FOREIGN KEY (product_id) REFERENCES Products(product_id)  
36 );
```

```

37 -- Insert sample data into Users table
38 INSERT INTO Users (user_id, first_name, last_name, email) VALUES
39 (1, 'Alice', 'Johnson', 'alice.johnson@example.com'),
40 (2, 'Bob', 'Smith', 'bob.smith@example.com');
41 -- Insert sample data into Products table
42 INSERT INTO Products (product_id, product_name, price, stock)
VALUES
43 (1, 'Laptop', 999.99, 10),
44 (2, 'Smartphone', 499.99, 20);
45 -- Insert sample data into Orders table
46 INSERT INTO Orders (order_id, user_id, order_date,
total_amount) VALUES
47 (1, 1, '2023-10-01', 1499.98),
48 (2, 2, '2023-10-02', 499.99);

49 -- Insert sample data into OrderItems table
50 INSERT INTO OrderItems (order_item_id, order_id, product_id,
quantity, price) VALUES
51 (1, 1, 1, 1, 999.99),
52 (2, 1, 2, 1, 499.99),
53 (3, 2, 2, 1, 499.99);
54 -- Query to retrieve all users and their orders
55 SELECT u.first_name, u.last_name, o.order_date, o.total_amount
56 FROM Users u
57 JOIN Orders o ON u.user_id = o.user_id;
58 -- Query to retrieve all products and their total sales
59 SELECT p.product_name, SUM(oi.quantity) AS total_quantity_sold,
SUM(oi.price * oi.quantity) AS total_sales
60 FROM Products p
61 JOIN OrderItems oi ON p.product_id = oi.product_id
62 GROUP BY p.product_name;
63 -- Query to retrieve the top-selling products
64 SELECT p.product_name, SUM(oi.quantity) AS total_quantity_sold
65 FROM Products p
66 JOIN OrderItems oi ON p.product_id = oi.product_id
67 GROUP BY p.product_name
68 ORDER BY total_quantity_sold DESC;

```

Output:

```
Run SQLite
54 FROM Users u
55 JOIN Orders o ON u.user_id = o.user_id;
56 -- Query to retrieve all products and their total sales
57 SELECT p.product_name, SUM(oi.quantity) AS total_quantity_sold, SUM(oi.price * oi.quantity) AS total_sales
58 FROM Products p
59 JOIN OrderItems oi ON p.product_id = oi.product_id
60 GROUP BY p.product_name;
61 -- Query to retrieve the top-selling products
62 SELECT p.product_name, SUM(oi.quantity) AS total_quantity_sold
63 FROM Products p
64 JOIN OrderItems oi ON p.product_id = oi.product_id
65 GROUP BY p.product_name
66 ORDER BY total_quantity_sold DESC;
```

first_name	last_name	order_date	total_amount
Alice	Johnson	2023-10-01	1499.98
Bob	Smith	2023-10-02	499.99

Explanation:

The e-commerce database includes Users, Products, Orders, and OrderDetails tables. OrderDetails acts as a linking table to handle multiple products per order. Queries retrieve user orders, identify top-selling products, and calculate revenue using aggregation. The schema follows normalization by separating product and order data, preventing duplication. This structure supports scalable and efficient transaction processing.

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

```

1  --generate University exam database SQL schema and queries for
   results and average marks
2  -- Create the University database
3  CREATE DATABASE UniversityDB;
4  USE UniversityDB;
5  -- Create Students table
6  CREATE TABLE Students (
7      student_id INT PRIMARY KEY,
8      first_name VARCHAR(50) NOT NULL,
9      last_name VARCHAR(50) NOT NULL,
10     email VARCHAR(100) UNIQUE NOT NULL
11 );
12 -- Create Courses table
13 CREATE TABLE Courses (
14     course_id INT PRIMARY KEY,
15     course_name VARCHAR(100) NOT NULL,
16     credits INT NOT NULL
17 );
18 -- Create Exams table
19 CREATE TABLE Exams (
20     exam_id INT PRIMARY KEY,
21     course_id INT,
22     exam_date DATE,
23     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
24 );
25 -- Create Results table
26 CREATE TABLE Results (
27     result_id INT PRIMARY KEY,
28     student_id INT,
29     exam_id INT,
30     marks INT NOT NULL,
31     FOREIGN KEY (student_id) REFERENCES Students(student_id),
32     FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
33 );

```

```
34 -- Insert sample data into Students table
35 INSERT INTO Students (student_id, first_name, last_name, email)
VALUES
36 (1, 'Alice', 'Johnson', 'alice.johnson@example.com'),
37 (2, 'Bob', 'Smith', 'bob.smith@example.com');
38 -- Insert sample data into Courses table
39 INSERT INTO Courses (course_id, course_name, credits) VALUES
40 (101, 'Introduction to Computer Science', 3),
41 (102, 'Database Systems', 4);
42 -- Insert sample data into Exams table
43 INSERT INTO Exams (exam_id, course_id, exam_date) VALUES
44 (1, 101, '2023-10-01'),
45 (2, 102, '2023-10-02');
46 -- Insert sample data into Results table
47 INSERT INTO Results (result_id, student_id, exam_id, marks)
VALUES
48 (1, 1, 1, 85),
49 (2, 2, 2, 90);
50 -- Query to retrieve all students and their exam results
```

```
50 -- Query to retrieve all students and their exam results
51 SELECT s.first_name, s.last_name, c.course_name, r.marks
52 FROM Students s
53 JOIN Results r ON s.student_id = r.student_id
54 JOIN Exams e ON r.exam_id = e.exam_id
55 JOIN Courses c ON e.course_id = c.course_id;
56 -- Query to calculate the average marks for each course
57 SELECT c.course_name, AVG(r.marks) AS average_marks
58 FROM Courses c
59 JOIN Exams e ON c.course_id = e.course_id
60 JOIN Results r ON e.exam_id = r.exam_id
61 GROUP BY c.course_name;
62 -- Query to retrieve the highest marks for each course
63 SELECT c.course_name, MAX(r.marks) AS highest_marks
64 FROM Courses c
65 JOIN Exams e ON c.course_id = e.course_id
66 JOIN Results r ON e.exam_id = r.exam_id
67 GROUP BY c.course_name;
```

Output:

```
54 JOIN Courses C ON e.course_id = C.course_id;
55 -- Query to calculate the average marks for each course
56 SELECT C.course_name, AVG(r.marks) AS average_marks
57 FROM Courses C
58 JOIN Exams e ON C.course_id = e.course_id
59 JOIN Results r ON e.exam_id = r.exam_id
60 GROUP BY C.course_name;
61 -- Query to retrieve the highest marks for each course
62 SELECT C.course_name, MAX(r.marks) AS highest_marks
63 FROM Courses C
64 JOIN Exams e ON C.course_id = e.course_id
65 JOIN Results r ON e.exam_id = r.exam_id
66 GROUP BY C.course_name;
```

first_name	last_name	course_name	marks
Alice	Johnson	Introduction to Computer Science	85
Bob	Smith	Database Systems	90

Explanation:

The examination system uses Students, Subjects, Exams, and Results tables to manage academic records. Results link students with exams and store marks. JOIN queries retrieve subject-wise performance, while aggregation functions calculate average marks per subject. The schema ensures referential integrity using foreign keys and supports efficient academic analysis. This normalized design avoids redundancy and maintains data accuracy.